

Clasa list din Python — operatori și metode

Fișă de lucru — Informatică, clasa a IX-a (matematică-informatică, regim intensiv)

Nivel de vârstă și utilitate

Clasa a IX-a, filiera teoretică, profil real, specializarea matematică-informatică, regim intensiv de predare a informaticii.

Materialul recapitulează și extinde lucrul cu liste (clasa list), structura de date fundamentală pentru organizarea liniară a datelor în Python, punând accent pe operatorii specifici și pe metodele uzuale de prelucrare.

1. Noțiuni teoretice

O listă (list) este o colecție ordonată și mutabilă de elemente, care pot fi accesate prin poziție (index), începând de la 0.

- Operatori specifici: [] pentru acces la un element, in / not in pentru testarea apartenenței, + pentru concatenare, * pentru multiplicare (repetare), ==, != și <, > pentru comparare.
- Metode de bază: append(x) adaugă un element la final, insert(i, x) inserează la poziția i, pop(i) elimină și returnează elementul de la poziția i, remove(x) elimină prima apariție a valorii x.
- Alte metode utile: sort() ordonează lista, reverse() inversează ordinea, copy() creează o copie, clear() golește lista, count(x) numără aparițiile lui x, index(x) găsește poziția lui x.

Tabel de referință — operatori specifici clasei list

Element	Exemplu	Descriere
<code>lista[i]</code>	<code>note[0]</code>	Accesează elementul de pe poziția i (indexare de la 0).
<code>in</code>	<code>'x' in lista</code>	Testează dacă valoarea există în listă; rezultat True/False.
<code>not in</code>	<code>'x' not in lista</code>	Testează dacă valoarea NU există în listă.
<code>+</code>	<code>lista1 + lista2</code>	Concatenează două liste, rezultând o listă nouă.
<code>*</code>	<code>lista * 3</code>	Repetă elementele listei de un număr dat de ori.
<code>== / !=</code>	<code>lista1 == lista2</code>	Compară dacă listele au aceleași elemente, în aceeași ordine.
<code>< / ></code>	<code>lista1 < lista2</code>	Compară listele element cu element (ordine lexicografică).

Tabel de referință — metode ale clasei list

Element	Exemplu	Descriere
<code>append(x)</code>	<code>l.append(5)</code>	Adaugă elementul x la finalul listei.
<code>insert(i, x)</code>	<code>l.insert(0, 5)</code>	Inserează x pe poziția i, fără a șterge celelalte elemente.
<code>pop(i)</code>	<code>l.pop()</code>	Elimină și returnează elementul de pe poziția i (implicit ultimul).
<code>remove(x)</code>	<code>l.remove(5)</code>	Elimină prima apariție a valorii x din listă.
<code>sort()</code>	<code>l.sort()</code>	Ordonează elementele crescător, modificând lista inițială.
<code>reverse()</code>	<code>l.reverse()</code>	Inversează ordinea elementelor, modificând lista inițială.

copy()	<code>l2 = l1.copy()</code>	Returnează o copie independentă (superficială) a listei.
clear()	<code>l1.clear()</code>	Elimină toate elementele; lista rămâne goală.
count(x)	<code>l1.count(5)</code>	Numără de câte ori apare valoarea x în listă.
index(x)	<code>l1.index(5)</code>	Returnează poziția primei apariții a valorii x.
extend(alta)	<code>l1.extend(l2)</code>	Adaugă, pe rând, toate elementele altei liste la final.

2. Exemple rezolvate

Exemplul 1 — gestionarea unei liste de cumpărături:

```
cumparaturi = ['pâine', 'lapte']
cumparaturi.append('mere')
cumparaturi.insert(0, 'apă')
print(cumparaturi)    # ['apă', 'pâine', 'lapte', 'mere']
print(len(cumparaturi)) # 4
```

Exemplul 2 — sortarea și inversarea unei liste de note:

```
note = [7, 10, 8, 9, 6]
note.sort()
print(note)          # [6, 7, 8, 9, 10]
note.reverse()
print(note)          # [10, 9, 8, 7, 6]
```

Exemplul 3 — numărare și căutare într-o listă:

```
raspunsuri = ['a', 'b', 'a', 'c', 'a', 'b']
print(raspunsuri.count('a')) # 3
print(raspunsuri.index('c')) # 3 (prima poziție a lui 'c')
```

3. Exerciții propuse

1. Scrieți un program care citește 5 nume de la tastatură, le adaugă într-o listă și afișează lista sortată alfabetic (metoda `sort()`).
2. Având lista `note = [4, 8, 10, 5, 9, 3]`, scrieți instrucțiuni care elimină toate notele mai mici decât 5 și afișează lista rezultată.
3. Scrieți o funcție care primește o listă de cuvinte și returnează o nouă listă, copie a celei inițiale, dar fără duplicate (folosind `count()` sau structuri de test).
4. Folosind operatorii `+` și `*`, construiți lista `[0, 0, 0, 1, 2, 3, 0, 0, 0]` pornind de la liste mai scurte.

4. Barem de corectare

Cerință	Punctaj
Exercițiul 1 — citire corectă (0,5p) + sortare și afișare (1p)	1,5p
Exercițiul 2 — parcurgere/filtrare corectă (1p) + afișare (0,5p)	1,5p
Exercițiul 3 — funcție corectă, fără duplicate (2p)	2p
Exercițiul 4 — folosirea corectă a operatorilor + și *	1,5p
Cod comentat, denumiri sugestive, testare cu exemple	1,5p
Total	10p (echivalat: 1p din examenul scris)

5. Bibliografie și webografie

- Python Software Foundation — „More on Lists”, docs.python.org/3/tutorial/datastructures.html
- w3schools.com — „Python Lists”, w3schools.com/python/python_lists.asp
- Programa școlară Informatică, clasa a IX-a, curriculum de specialitate, Anexa nr. 49, OMEC 2025–2026.